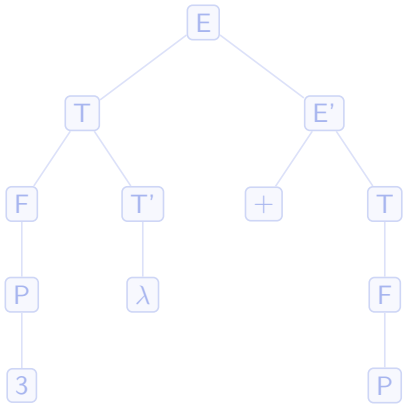
Universidad de Antioquia
Facultad de Ingeniería
Teoría de Lenguajes

Práctica #9

Reconocimiento Descendente

Analizador sintáctico descendente recursivo LL(1) para una calculadora simple, con evaluación, diagnóstico de errores e interfaz gráfica animada



Autores

Alejandro Botero Arbeláez
C.C. 1035328211

Esteban Luna Seña

Lenguaje

Python 3
tkinter (estándar)

Fecha

11 de noviembre
de 2026

Resumen

Este informe documenta el diseño, la implementación y la validación de un *analizador sintáctico descendente recursivo* para el lenguaje de una calculadora simple con números reales, los operadores binarios $+$, $-$, $*$, $/$, $\hat{}$ (potencia), el menos unario y paréntesis. Partiendo de una gramática ambigua se obtiene, mediante estratificación por precedencia, eliminación de la recursividad por la izquierda y factorización, una gramática LL(1) cuyos conjuntos Primero y Siguiente se calculan automáticamente y cuya tabla de análisis se verifica libre de conflictos. El reconocedor —un procedimiento por no terminal— acepta las cadenas válidas, construye el árbol de derivación y el árbol de sintaxis abstracta (AST), y evalúa la expresión respetando precedencia y asociatividad. Ante una entrada inválida informa la *clase* del error (léxico, sintáctico o semántico), su *posición* y los símbolos que se esperaban. Se acompaña de una interfaz gráfica que anima paso a paso el reconocimiento: puntero de preanálisis, producción activa, traza de llamadas y crecimiento del árbol. Una batería de 26 casos de prueba y un experimento sobre 140 expresiones aleatorias confirman la corrección del analizador y su costo lineal en el tamaño de la entrada.

Palabras clave: análisis sintáctico descendente, gramáticas LL(1), conjuntos Primero y Siguiente, descenso recursivo, árbol de sintaxis abstracta, recuperación y diagnóstico de errores.

Contenido

1. Introducción **3**
- 1.1. Objetivos . . . 3
- 1.2. Entregables . . . 3

2. Marco teórico **3**
- 2.1. Gramáticas libres de contexto y derivaciones . . . 4
- 2.2. Análisis descendente y gramáticas LL(1) . . . 4
- 2.3. Descenso recursivo . . . 4

3. Diseño del lenguaje **4**
- 3.1. Componentes léxicos . . . 5
- 3.2. De la gramática ambigua a la gramática LL(1) . . . 5
- 3.3. Precedencia y asociatividad . . . 6

4. Análisis LL(1) de la gramática **7**
- 4.1. Conjuntos PRIMERO y SIGUIENTE . . . 7
- 4.2. Tabla de análisis predictivo . . . 7

5. Implementación **8**
- 5.1. Arquitectura . . . 8
- 5.2. Analizador léxico . . . 8
- 5.3. Analizador sintáctico: un procedimiento por no terminal . . . 9
- 5.4. Elección de λ y detección temprana de errores . . . 11
- 5.5. Evaluación sobre el AST . . . 12
- 5.6. Clasificación de errores . . . 14
- 5.7. Interfaz gráfica . . . 14

6. Ejemplos desarrollados **15**
- 6.1. Ejemplo 1: `3 + 4 * 2` (precedencia) . . . 15
- 6.2. Ejemplo 2: asociatividad de la potencia y menos unario . . . 18
- 6.3. Ejemplo 3: expresión compuesta . . . 18
- 6.4. Ejemplo 4: error sintáctico `3 + * 4` . . . 19
- 6.5. Ejemplo 5: errores léxico y semántico . . . 20

7. Pruebas y resultados **20**
- 7.1. Batería de pruebas . . . 20
- 7.2. Costo del reconocimiento . . . 21

8. Manual de uso **22**

9. Conclusiones **23**

Referencias **23**

A. Cálculo de PRIMERO y SIGUIENTE **23**

1 Introducción

El análisis sintáctico es la fase del compilador que decide si una secuencia de componentes léxicos pertenece al lenguaje generado por una gramática libre de contexto (GIC) y, en caso afirmativo, recupera su estructura jerárquica. Entre las estrategias existentes, el *análisis descendente* construye el árbol de derivación desde la raíz hacia las hojas, reproduciendo una *derivación por la izquierda* de la cadena. Cuando la gramática es LL(1), la decisión de qué producción aplicar puede tomarse mirando un único símbolo de la entrada, lo que permite escribir el analizador directamente como un conjunto de procedimientos mutuamente recursivos: el *descenso recursivo*.

Enunciado

Práctica #9: Reconocimiento Descendente. Elaborar un analizador sintáctico que responda a la sintaxis de una calculadora simple. Además de admitir los tokens correctos, presentará también el resultado de la operación. Cuando ocurra un error de sintaxis deberá indicar cuál es el error. *Lenguajes:* C, C++, C#, JavaScript, Java, Kotlin o Python.

1.1 Objetivos

General. Construir un reconocedor descendente recursivo para expresiones aritméticas que acepte las cadenas del lenguaje, calcule su valor y diagnostique con precisión las cadenas inválidas.

Específicos.

1. Definir los componentes léxicos de la calculadora e implementar su analizador léxico.
2. Transformar una gramática natural (ambigua y recursiva por la izquierda) en una gramática LL(1) equivalente que codifique precedencia y asociatividad.
3. Calcular PRIMERO y SIGUIENTE, construir la tabla de análisis y verificar la ausencia de conflictos.
4. Implementar un procedimiento por no terminal que construya el árbol de derivación y el AST.
5. Evaluar la expresión y reportar errores léxicos, sintácticos y semánticos con su posición.
6. Visualizar el proceso de reconocimiento de forma didáctica mediante una interfaz gráfica.

1.2 Entregables

Archivo	Contenido
`analizador.py`	Núcleo: analizador léxico, analizador descendente recursivo, cálculo de PRIMERO/SIGUIENTE, evaluador y modo consola.
`app.py`	Interfaz gráfica animada (solo biblioteca estándar: `tkinter`).
`informe/`	Este informe; `generar.py` produce sus tablas, trazas y árboles ejecutando el analizador.

2 Marco teórico

2.1 Gramáticas libres de contexto y derivaciones

Una GIC es una cuádrupla $G = (V, \Sigma, P, S)$ con no terminales V, terminales Σ, producciones $P \subseteq V \times (V \cup \Sigma)^*$ y símbolo inicial S. Una *derivación por la izquierda* ($\overset{\text{lm}}{\Longrightarrow}$) reemplaza en cada paso el no terminal situado más a la izquierda. Una gramática es *ambigua* si alguna cadena admite dos árboles de derivación distintos; en una calculadora esto se traduce en dos valores posibles para la misma expresión.

2.2 Análisis descendente y gramáticas LL(1)

Un analizador descendente parte de S y, leyendo la entrada de izquierda a derecha, elige qué alternativa expandir. En una gramática LL(1) (*Left-to-right, Leftmost derivation, 1 símbolo de preanálisis*) esa elección es determinista. Para formalizarla se definen:

- $\textsc{Primero}(\alpha)$: terminales que pueden iniciar una cadena derivada de α; incluye λ si $\alpha \Rightarrow^* \lambda$.
- $\textsc{Siguiente}(A)$: terminales que pueden aparecer inmediatamente a la derecha de A en alguna forma sentencial; incluye el fin de entrada `$` si A puede terminar la cadena.

> **Condición LL(1).** Para cada par de alternativas $A \to \alpha \mid \beta$: (i) $\textsc{Primero}(\alpha) \cap \textsc{Primero}(\beta) = \varnothing$, y (ii) si $\beta \Rightarrow^* \lambda$, entonces $\textsc{Primero}(\alpha) \cap \textsc{Siguiente}(A) = \varnothing$. Equivalentemente, cada celda $M[A, a]$ de la tabla de análisis contiene a lo sumo una producción.

Dos transformaciones habituales llevan una gramática a esta forma:

- **Eliminación de la recursividad por la izquierda.** $A \to A\alpha \mid \beta$ se reemplaza por $A \to \beta A'$ y $A' \to \alpha A' \mid \lambda$, que genera el mismo lenguaje $\beta\alpha^*$ sin que el analizador entre en un ciclo infinito.
- **Factorización por la izquierda.** $A \to \gamma\alpha \mid \gamma\beta$ se reemplaza por $A \to \gamma A'$ y $A' \to \alpha \mid \beta$, postergando la decisión hasta después del prefijo común.

2.3 Descenso recursivo

En el descenso recursivo cada no terminal A se implementa como un procedimiento que, según el token de preanálisis, elige la alternativa $A \to X_1 \cdots X_k$ y la recorre: si X_i es terminal se *empareja* (*match*) con la entrada; si es no terminal se invoca su procedimiento. La pila de llamadas del lenguaje de programación cumple el papel de la pila del autómata con pila del analizador predictivo, y la secuencia de expansiones es exactamente la derivación por la izquierda de la cadena.

3 Diseño del lenguaje

3.1 Componentes léxicos

Tabla 1: Componentes léxicos reconocidos por `tokenizar()`.

Token	Símbolo	Patrón	Observaciones
NUM	num	`[0-9]+(\.[0-9]*)?|\.[0-9]+`	Enteros o decimales: `7`, `3.5`, `.5`.
MAS	+	+	
MENOS	-	- | −	Binario o unario (lo decide la sintaxis).
POR	*	* | × | ·	
DIV	/	/ | ÷	
POT	^	^	Asociativo por la derecha.
PAR_IZQ, PAR_DER	( )	( )	
FIN	$	fin de texto	Marcador agregado por el analizador léxico.

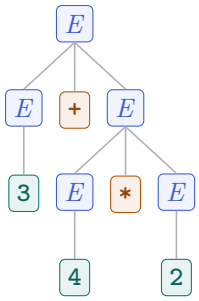
Los espacios se ignoran. Cualquier otro carácter, o un número con dos puntos decimales, produce un *error léxico* que detiene el proceso antes del análisis sintáctico.

3.2 De la gramática ambigua a la gramática LL(1)

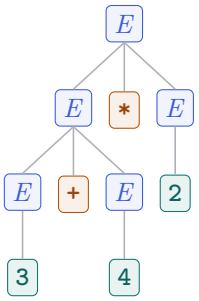
Paso 0: gramática natural. La forma más directa de describir la calculadora es

$$E \to E\,\texttt{+}\,E \mid E\,\texttt{-}\,E \mid E\,\texttt{*}\,E \mid E\,\texttt{/}\,E \mid E\,\texttt{\^{}}\,E \mid \texttt{-}\,E \mid \texttt{(}\,E\,\texttt{)} \mid \texttt{num}$$

que es **ambigua**: la cadena `3 + 4 * 2` tiene los dos árboles de la figura 1, con valores distintos. Además es recursiva por la izquierda, por lo que un analizador descendente no podría usarla.



(a) $3 + (4 \cdot 2) = \mathbf{11}$ ✓



(b) $(3 + 4) \cdot 2 = \mathbf{14}$ ×

Figura 1: Dos árboles de derivación para `3 + 4 * 2` con la gramática ambigua.

Paso 1: estratificación por precedencia. Se introduce un no terminal por nivel de precedencia; los operadores de menor precedencia quedan más cerca de la raíz:

$$E \to E\,\texttt{+}\,T \mid E\,\texttt{-}\,T \mid T$$
$$T \to T\,\texttt{*}\,F \mid T\,\texttt{/}\,F \mid F$$
$$F \to \texttt{-}\,F \mid P\,\texttt{\^{}}\,F \mid P$$
$$P \to \texttt{num} \mid \texttt{(}\,E\,\texttt{)}$$

La recursividad por la izquierda de E y T fija la asociatividad izquierda de $+, -, *, /$; la recursividad por la derecha de $P\,\texttt{\^{}}\,F$ fija la asociatividad derecha de la potencia. Esta gramática ya no es ambigua, pero aún no es LL(1).

Paso 2: eliminación de la recursividad izquierda. Con $A = E$, $\alpha \in \{\texttt{+}T, \texttt{-}T\}$, $\beta = T$ se obtiene $E \to T\,E'$ y $E' \to \texttt{+}\,T\,E' \mid \texttt{-}\,T\,E' \mid \lambda$; análogamente para T.

Paso 3: factorización. Las alternativas $P\,\texttt{\^{}}\,F$ y P comparten el prefijo P: se factoriza como $F \to P\,F'$ con $F' \to \texttt{\^{}}\,F \mid \lambda$.

Tabla 2: Gramática LL(1) final implementada (`GRAMATICA` en `analizador.py`).

$$\begin{aligned}
E &\to T\,E' \\
E' &\to \texttt{+}\,T\,E' \mid \texttt{-}\,T\,E' \mid \lambda \\
T &\to F\,T' \\
T' &\to \texttt{*}\,F\,T' \mid \texttt{/}\,F\,T' \mid \lambda \\
F &\to \texttt{-}\,F \mid P\,F' \\
F' &\to \texttt{\^{}}\,F \mid \lambda \\
P &\to \texttt{num} \mid \texttt{(}\,E\,\texttt{)}
\end{aligned}$$

3.3 Precedencia y asociatividad

Tabla 3: Precedencia (de menor a mayor) y asociatividad resultantes.

Nivel	Operadores	No terminal	Asociatividad	Ejemplo
1	`+ -` binarios	E, E'	izquierda	`10-4-3` $= (10-4)-3 = 3$
2	`* /`	T, T'	izquierda	`100/10/5` $= 2$
3	`-` unario	F	derecha (prefijo)	`-3` $= 3$
4	`^`	F'	derecha	`2^3^2` $= 2^{9} = 512$
5	`(...)`, `num`	P	—	`(3+4)*2` $= 14$

Nota Como `^` está por encima del menos unario, `-2^2` se interpreta como $-(2^2) = -4$, la convención matemática usual; y como $F' \to$ `^` F invoca a F, también se admiten exponentes negativos: `2^-1` $= 0.5$.

4 Análisis LL(1) de la gramática

4.1 Conjuntos PRIMERO y SIGUIENTE

Los conjuntos se calculan en `calcular_primero_siguiente()` por *iteración de punto fijo*: se recorren las producciones agregando símbolos hasta que ningún conjunto cambia (sección 5). El resultado, impreso directamente por el programa, es:

Tabla 4: Conjuntos PRIMERO y SIGUIENTE calculados por el programa.

No terminal	Primero	Siguiente
E	{ `num`, `(`, `-` }	{ `)`, `$` }
E'	{ `+`, `-`, λ }	{ `)`, `$` }
T	{ `num`, `(`, `-` }	{ `)`, `+`, `-`, `$` }
T'	{ `*`, `/`, λ }	{ `)`, `+`, `-`, `$` }
F	{ `num`, `(`, `-` }	{ `)`, `+`, `-`, `*`, `/`, `$` }
F'	{ `^`, λ }	{ `)`, `+`, `-`, `*`, `/`, `$` }
P	{ `num`, `(` }	{ `)`, `+`, `-`, `*`, `/`, `^`, `$` }

Cálculo comentado de algunos conjuntos.

- PRIMERO(P) = {`num`, `(`} directamente. Como $F \to$ `-`$F \mid PF'$, PRIMERO(F) = {`-`} $\cup$ PRIMERO(P), y por $T \to FT'$ y $E \to TE'$ este conjunto se propaga sin cambios a T y E.
- SIGUIENTE(E) = {`$`} por ser el símbolo inicial, más `)` por $P \to$ `(`E`)`.
- En $E \to T\,E'$: SIGUIENTE(T) $\supseteq$ PRIMERO(E') $\setminus \{\lambda\}$ = {`+`, `-`} y, como $E' \Rightarrow \lambda$, también SIGUIENTE(T) $\supseteq$ SIGUIENTE(E).
- Del mismo modo SIGUIENTE(P) = (PRIMERO(F') $\setminus \{\lambda\}$) $\cup$ SIGUIENTE(F) agrega `^` a todos los operadores: tras un operando puede venir cualquier operador, `)` o el fin.

4.2 Tabla de análisis predictivo

Para cada producción $A \to \alpha$ se coloca α en $M[A, a]$ para todo $a \in$ PRIMERO(α); si $\lambda \in$ PRIMERO(α), también en $M[A, b]$ para todo $b \in$ SIGUIENTE(A). El script `generar.py` construye la tabla y *aborta si encuentra dos producciones en la misma celda*: la tabla 5 se obtuvo sin conflictos (28 entradas), lo que demuestra que la gramática es LL(1). Una celda vacía significa error sintáctico.

Tabla 5: Tabla de análisis $M[A, a]$ (λ se muestra como alternativa vacía).

	num	(	)	+	-	*	/	^	$
E	$T\ E'$	$T\ E'$	error	error	$T\ E'$	error	error	error	error
E'	error	error	λ	+ $T\ E'$	- $T\ E'$	error	error	error	λ
T	$F\ T'$	$F\ T'$	error	error	$F\ T'$	error	error	error	error
T'	error	error	λ	λ	λ	* $F\ T'$	/ $F\ T'$	error	λ
F	$P\ F'$	$P\ F'$	error	error	- F	error	error	error	error
F'	error	error	λ	λ	λ	λ	λ	^ F	λ
P	num	(E)	error	error	error	error	error	error	error

Verificación de la condición LL(1). E': {+}, {-} y Siguiente(E') = {), $} son disjuntos. T': {*}, {/} y {), +, -, $} son disjuntos. F: {-} ∩ {num, (} = ∅. F': {^} ∩ Siguiente(F') = ∅. P: {num} ∩ {(} = ∅.

5 Implementación

5.1 Arquitectura

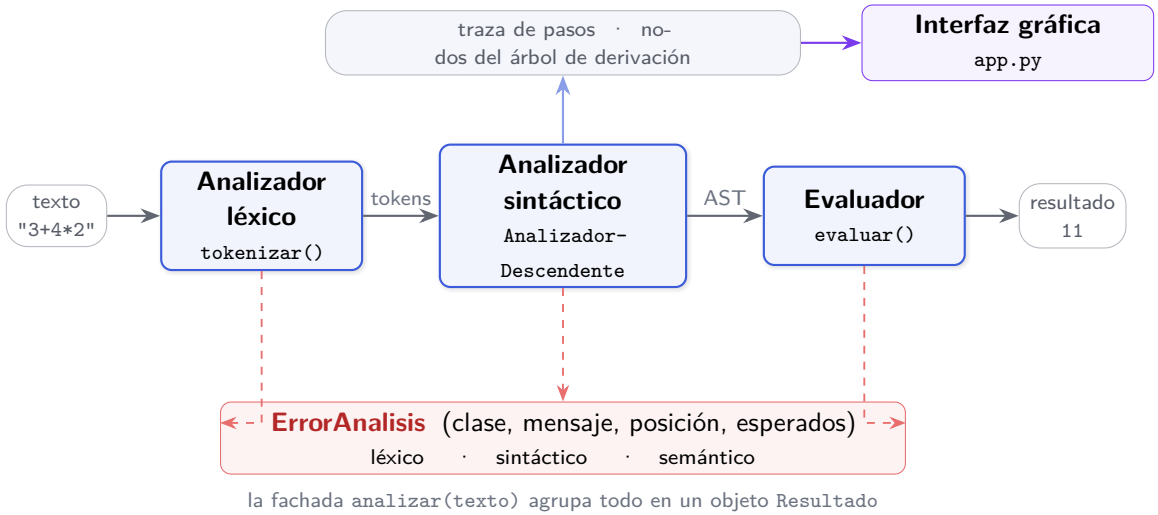


Figura 2: Arquitectura del sistema. Cada etapa puede producir un `ErrorAnalisis` localizado; la interfaz solo consume el `Resultado`, por lo que el núcleo es independiente de la presentación.

5.2 Analizador léxico

El analizador léxico recorre el texto una sola vez. Los números se reconocen de forma ávida (dígitos, punto opcional, dígitos); si tras un número aparece otro punto o una letra, se informa un número mal formado. Al terminar se añade el token `FIN`, que hace de `$`.

analizador.py — tokenizar()

```
def tokenizar(texto: str) -> tuple[list[Token], ErrorAnalisis | None]:
    """Devuelve (tokens, error). Si hay error léxico, la lista termina en un
    token INVALIDO para poder mostrar hasta dónde se reconoció."""
    tokens: list[Token] = []
```

```
    i, n = 0, len(texto)
    while i < n:
        c = texto[i]
        if c.isspace():
            i += 1
        elif c.isdigit() or (c == "." and i + 1 < n and texto[i + 1].isdigit()):
            j = i
            while j < n and texto[j].isdigit():
                j += 1
            if j < n and texto[j] == ".":
                j += 1
                while j < n and texto[j].isdigit():
                    j += 1
            if j < n and (texto[j] == "." or texto[j].isalpha()):
                k = j
                while k < n and (texto[k].isalnum() or texto[k] == "."):
                    k += 1
                tokens.append(Token(INVALIDO, texto[i:k], i))
                return tokens, ErrorAnalisis(
                    "léxico", f"Número mal formado: '{texto[i:k]}'", i, k - i)
            lex = texto[i:j]
            tokens.append(Token(NUM, lex, i, float(lex)))
            i = j
        elif c in SIMBOLOS:
            tokens.append(Token(SIMBOLOS[c], c, i))
            i += 1
        else:
            tokens.append(Token(INVALIDO, c, i))
            return tokens, ErrorAnalisis(
                "léxico", f"Carácter no reconocido '{c}' en la posición {i + 1}", i)
    tokens.append(Token(FIN, "", n))
    return tokens, None
```

5.3 Analizador sintáctico: un procedimiento por no terminal

La correspondencia entre la gramática y el código es uno a uno (tabla 6). Las llamadas entre procedimientos forman el grafo de la figura 3, cuyos ciclos son precisamente las recursiones de la gramática.

Tabla 6: Procedimientos del analizador y decisión que toma cada uno.

NT	Método	Decisión con el preanálisis a
E	`E()`	Siempre $T\ E'$.
E'	`Ep()`	$a =$ `+` o `-`: empareja, llama a `T()` y recursa; si no, λ *solo si* $a \in$ SIGUIENTE(E').
T	`T()`	Siempre $F\ T'$.
T'	`Tp()`	$a =$ `*` o `/`: empareja, llama a `F()` y recursa; si no, λ si $a \in$ SIGUIENTE(T').
F	`F()`	$a =$ `-`: menos unario; en otro caso $P\ F'$.
F'	`Fp()`	$a =$ `^`: empareja y llama a `F()` (asociatividad derecha); si no, λ.
P	`P()`	$a =$ `num`: lo empareja; $a =$ `(`: `(` `E()` `)`; en otro caso, error.

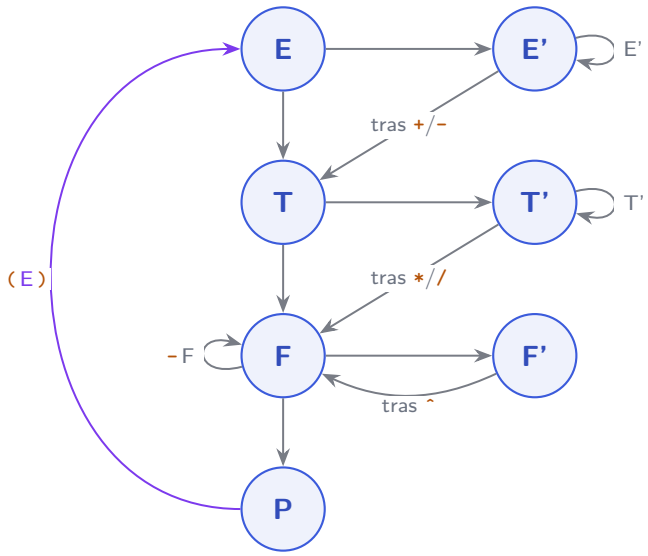


Figura 3: Grafo de llamadas entre procedimientos. Los lazos corresponden a la recursión de cola que reemplazó a la recursividad por la izquierda; el arco $P \to E$ genera el anidamiento con paréntesis.

Los procedimientos de E y E' ilustran el patrón completo. Nótese el parámetro `izq`: es un *atributo heredado* que transporta el subárbol ya reconocido a la izquierda.

analizador.py — E() y Ep()

```
def E(self, padre, nivel) -> NodoAST:                    # E → T E'
    n = self._expandir(padre, "E", 0, nivel)
    izq = self.T(n, nivel + 1)
    return self.Ep(n, nivel + 1, izq)

def Ep(self, padre, nivel, izq) -> NodoAST:              # E' → + T E' | − T E' | λ
    if self.la.tipo in (MAS, MENOS):
        alt = 0 if self.la.tipo == MAS else 1
        n = self._expandir(padre, "E'", alt, nivel)
        op = self._coincidir(n, self.la.tipo, nivel + 1)
```

```
        der = self.T(n, nivel + 1)
        return self.Ep(n, nivel + 1, NodoAST(op.simbolo, "bin", [izq, der], op))
    self._elegir_lambda("E'", 2, padre, nivel)
    return izq
```

Recuperar la asociatividad izquierda. La gramática LL(1) es recursiva por la *derecha*, de modo que su árbol de derivación "cuelga" hacia la derecha. Si el AST se construyera copiando esa forma, `10-4-3` se evaluaría como $10 - (4 - 3) = 9$. Pasando `izq` hacia abajo, `Ep()` arma el nodo `(izq op der)` *antes* de recursar, y el AST queda inclinado a la izquierda (figura 4), con el valor correcto 3.

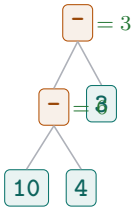


Figura 4: AST de `10 - 4 - 3`: asociatividad izquierda recuperada mediante el atributo heredado.

La potencia, en cambio, se asocia por la derecha de forma natural, porque F' invoca a F:

analizador.py — F() y Fp()

```
def F(self, padre, nivel) -> NodoAST:                    # F → − F | P F'
    if self.la.tipo == MENOS:
        n = self._expandir(padre, "F", 0, nivel)
        op = self._coincidir(n, MENOS, nivel + 1)
        return NodoAST("−", "neg", [self.F(n, nivel + 1)], op)
    n = self._expandir(padre, "F", 1, nivel)
    base = self.P(n, nivel + 1)
    return self.Fp(n, nivel + 1, base)

def Fp(self, padre, nivel, base) -> NodoAST:             # F' → ^ F | λ
    if self.la.tipo == POT:
        n = self._expandir(padre, "F'", 0, nivel)
        op = self._coincidir(n, POT, nivel + 1)
        exp = self.F(n, nivel + 1)
        return NodoAST("^", "bin", [base, exp], op)
    self._elegir_lambda("F'", 1, padre, nivel)
    return base
```

5.4 Elección de λ y detección temprana de errores

Un analizador descendente ingenuo elige λ siempre que no reconoce el preanálisis, y el error aparece más tarde, lejos de su causa. Aquí la alternativa vacía se elige **solo si el preanálisis pertenece a Siguiente** del no terminal (exactamente la regla de la tabla 5); de lo contrario el error se reporta en ese mismo punto, con los símbolos esperados:

analizador.py — _elegir_lambda()

```python
def _elegir_lambda(self, a, alt_lambda, padre, nivel) -> None:
    if self.la.simbolo in SIGUIENTE[a]:
        self._expandir(padre, a, alt_lambda, nivel)
        return
    # el preanálisis no puede seguir a este no terminal: falta un operador
    n = self._nodo(padre, a, "nt")
    prev = self.previo.lexema if self.previo else ""
    que = "'('" if self.la.tipo == PAR_IZQ else f"'{self.la.lexema}'"
    esperados = [s for s in PRIMERO[a] if s != "λ"] + SIGUIENTE[a]
    self._error(n, nivel, f"Falta un operador entre '{prev}' y {que}", esperados)
```

analizador.py — P(): operandos, paréntesis y diagnóstico

```python
def P(self, padre, nivel) -> NodoAST:                    # P → num | ( E )
    if self.la.tipo == NUM:
        n = self._expandir(padre, "P", 0, nivel)
        tok = self._coincidir(n, NUM, nivel + 1)
        return NodoAST(tok.lexema, "num", [], tok, tok.valor)
    if self.la.tipo == PAR_IZQ:
        n = self._expandir(padre, "P", 1, nivel)
        abre = self._coincidir(n, PAR_IZQ, nivel + 1)
        e = self.E(n, nivel + 1)
        if self.la.tipo != PAR_DER:
            self._error(n, nivel + 1,
                        f"Falta ')' para cerrar el '(' de la posición {abre.pos + 1}", [")
"])
        self._coincidir(n, PAR_DER, nivel + 1)
        return e
    # no hay operando donde debería haberlo
    n = self._nodo(padre, "P", "nt")
    la, prev = self.la, self.previo
    if la.tipo == FIN:
        msg = ("La expresión está vacía" if prev is None else
               f"Expresión incompleta: falta un operando después de '{prev.lexema}'")
    elif la.tipo == PAR_DER:
        msg = ("Paréntesis vacíos: se esperaba una expresión dentro de '( )'"
               if prev and prev.tipo == PAR_IZQ else
               "Se esperaba un número o '(' antes de ')'")
    elif prev is None:
        msg = f"La expresión no puede comenzar con el operador '{la.lexema}'"
    else:
        msg = f"Operador '{la.lexema}' inesperado después de '{prev.lexema}': falta un
operando"
    self._error(n, nivel, msg, PRIMERO["F"])
```

5.5 Evaluación sobre el AST

El evaluador recorre el AST en *postorden*; cada operación se registra para mostrarla al usuario. Los errores semánticos (división por cero, potencias indefinidas o no reales, desbordamiento) se distinguen de los sintácticos: la cadena *pertenece* al lenguaje, pero su valor no está definido.

analizador.py — evaluar()

```python
def evaluar(nodo: NodoAST, pasos: list[str]) -> float:
    """Recorrido en postorden; registra cada operación en `pasos`."""
    if nodo.clase == "num":
        return nodo.valor
    if nodo.clase == "neg":
        a = evaluar(nodo.hijos[0], pasos)
        nodo.valor = -a
        pasos.append(f"−({formatear(a)}) = {formatear(nodo.valor)}")
        return nodo.valor
    a = evaluar(nodo.hijos[0], pasos)
    b = evaluar(nodo.hijos[1], pasos)
    op, tok = nodo.etiqueta, nodo.token

    def fallo(msg):
        raise ErrorAnalisis("semántico", msg, tok.pos, 1)

    if op == "+":
        r = a + b
    elif op == "−":
        r = a - b
    elif op == "*":
        r = a * b
    elif op == "/":
        if b == 0:
            fallo(f"División por cero: {formatear(a)} / 0")
        r = a / b
    else:
        if a == 0 and b < 0:
            fallo(f"Potencia indefinida: 0 ^ {formatear(b)}")
        if a < 0 and b != int(b):
            fallo(f"Potencia no real: {formatear(a)} ^ {formatear(b)}")
        try:
            r = math.pow(a, b)
        except OverflowError:
            r = math.inf
    if math.isinf(r) or math.isnan(r):
        fallo("Desbordamiento: el resultado es demasiado grande")
    nodo.valor = r
    pasos.append(f"{formatear(a)} {op} {formatear(b)} = {formatear(r)}")
    return r
```

5.6 Clasificación de errores

Tabla 7: Errores detectados, dónde se detectan y mensaje que producen.

Clase	Situación	Detectado en	Mensaje (ejemplo)
léxico	Carácter fuera del alfabeto	`tokenizar()`	Carácter no reconocido `'$'` en la posición 3
léxico	Número con dos puntos	`tokenizar()`	Número mal formado: `'1.5.2'`
sintáctico	Falta un operando	`P()`	Operador `'*'` inesperado después de `'+'`: falta un operando
sintáctico	Falta un operador	`_elegir_lambda()`	Falta un operador entre `'3'` y `'4'`
sintáctico	Paréntesis sin cerrar	`P()` tras `E()`	Falta `')'` para cerrar el `'('` de la posición 1
sintáctico	Paréntesis sin abrir	`analizar()`	Paréntesis `')'` sin abrir en la posición 6
sintáctico	Entrada vacía o truncada	`P()`	Expresión incompleta: falta un operando después de `'+'`
semántico	Divisor nulo	`evaluar()`	División por cero: `8 / 0`
semántico	0^{-n}, $(-a)^{p/q}$	`evaluar()`	Potencia no real: `-8 ^ 0.5`
semántico	Resultado infinito	`evaluar()`	Desbordamiento: el resultado es demasiado grande

5.7 Interfaz gráfica

`app.py` presenta el proceso completo en una sola ventana (figura 5) y lo **anima paso a paso**: las fichas de tokens aparecen escalonadas y un puntero de *preanálisis* se desliza sobre la entrada; la producción aplicada se resalta en la tabla de gramática; la traza muestra cada llamada con su profundidad; y el árbol de derivación crece nodo a nodo con un anillo sobre el nodo actual. Al terminar se muestra el resultado con los pasos de evaluación, o el error con la posición subrayada, el campo de entrada vibra y el token culpable se marca en rojo. Una pestaña alterna con el AST, anotado con el valor de cada subexpresión.

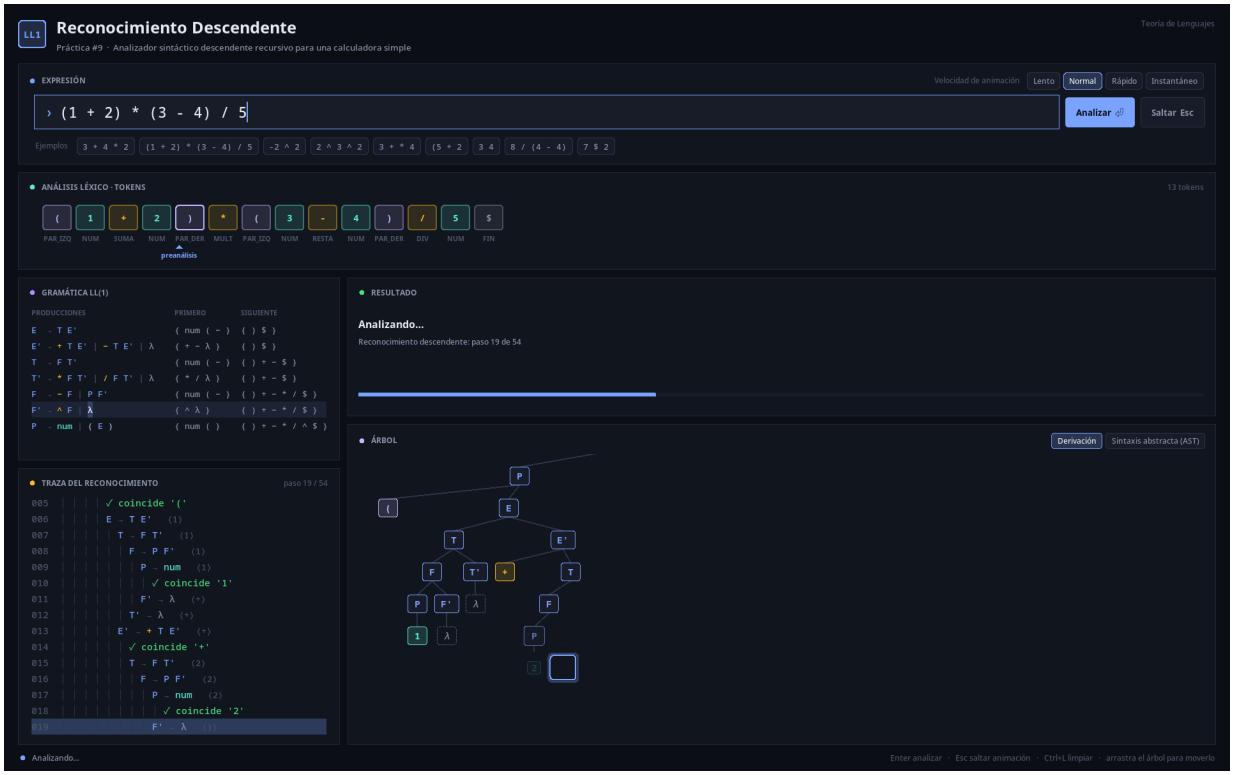
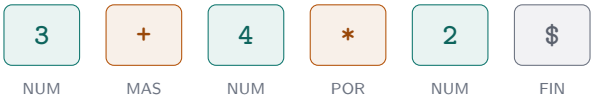


Figura 5: Interfaz durante el reconocimiento de `(1 + 2) * (3 - 4) / 5`, en el paso 19 de 54: el puntero de preanálisis está sobre `)`, la gramática resalta $F' \to \lambda$ y el árbol se está construyendo.

6 Ejemplos desarrollados

6.1 Ejemplo 1: `3 + 4 * 2` (precedencia)

Análisis léxico.



Traza del reconocimiento. La tabla 8 es la salida del analizador: cada fila es una expansión o un emparejamiento, sangrada según la profundidad de recursión. La columna de preanálisis muestra el único token usado para decidir.

Tabla 8: Traza del reconocimiento descendente de `3 + 4 * 2` (22 pasos).

#	Nivel	Acción	Preanálisis	Entrada restante
1	0	$E \to T\ E'$	3	3 + 4 * 2 $
2	1	$T \to F\ T'$	3	3 + 4 * 2 $
3	2	$F \to P\ F'$	3	3 + 4 * 2 $
4	3	$P \to$ num	3	3 + 4 * 2 $
5	4	✓ coincide 3	3	+ 4 * 2 $
6	3	$F' \to \lambda$	+	+ 4 * 2 $
7	2	$T' \to \lambda$	+	+ 4 * 2 $
8	1	$E' \to +\ T\ E'$	+	+ 4 * 2 $

#	Nivel	Acción	Preanálisis	Entrada restante
9	2	✓ coincide `+`	`+`	`4 * 2 $`
10	2	$T \to F\ T'$	`4`	`4 * 2 $`
11	3	$F \to P\ F'$	`4`	`4 * 2 $`
12	4	$P \to$ `num`	`4`	`4 * 2 $`
13	5	✓ coincide `4`	`4`	`* 2 $`
14	4	$F' \to \lambda$	`*`	`* 2 $`
15	3	$T' \to$ `*` $F\ T'$	`*`	`* 2 $`
16	4	✓ coincide `*`	`*`	`2 $`
17	4	$F \to P\ F'$	`2`	`2 $`
18	5	$P \to$ `num`	`2`	`2 $`
19	6	✓ coincide `2`	`2`	`$`
20	5	$F' \to \lambda$	`$`	`$`
21	4	$T' \to \lambda$	`$`	`$`
22	2	$E' \to \lambda$	`$`	`$`

Derivación por la izquierda. Leyendo solo las expansiones de la traza se obtiene, en orden, la derivación más a la izquierda de la cadena:

0	E	9	$\Rightarrow$ `num` `+` $P\ F'\ T'\ E'$
1	$\Rightarrow T\ E'$	10	$\Rightarrow$ `num` `+` `num` $F'\ T'\ E'$
2	$\Rightarrow F\ T'\ E'$	11	$\Rightarrow$ `num` `+` `num` $T'\ E'$
3	$\Rightarrow P\ F'\ T'\ E'$	12	$\Rightarrow$ `num` `+` `num` `*` $F\ T'\ E'$
4	$\Rightarrow$ `num` $F'\ T'\ E'$	13	$\Rightarrow$ `num` `+` `num` `*` $P\ F'\ T'\ E'$
5	$\Rightarrow$ `num` $T'\ E'$	14	$\Rightarrow$ `num` `+` `num` `*` `num` $F'\ T'\ E'$
6	$\Rightarrow$ `num` E'	15	$\Rightarrow$ `num` `+` `num` `*` `num` $T'\ E'$
7	$\Rightarrow$ `num` `+` $T\ E'$	16	$\Rightarrow$ `num` `+` `num` `*` `num` E'
8	$\Rightarrow$ `num` `+` $F\ T'\ E'$	17	$\Rightarrow$ `num` `+` `num` `*` `num`

Al final, cada `num` corresponde, en orden, a los lexemas `3`, `4` y `2`.

Árbol de derivación y AST. El árbol de derivación (figura 6) contiene todos los símbolos de la gramática, incluidas las producciones vacías; el AST (figura 7) conserva solo la estructura de operadores, y en él se ve que `*` queda más profundo que `+`: se evalúa primero.

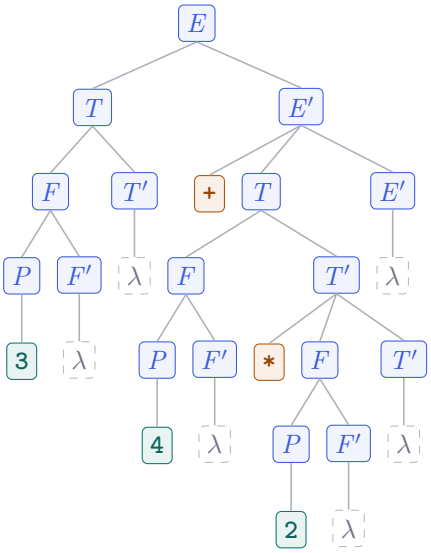


Figura 6: Árbol de derivación de `3 + 4 * 2` generado por el analizador (λ en líneas punteadas).

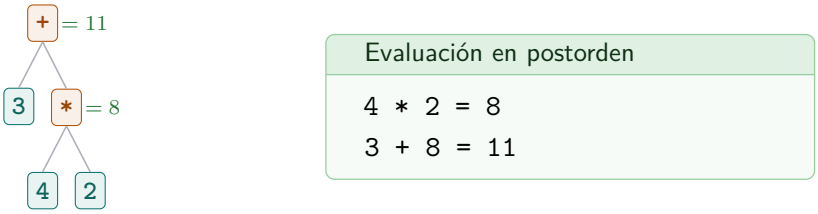


Figura 7: AST de `3 + 4 * 2` anotado con el valor de cada subexpresión. Resultado: **11**.

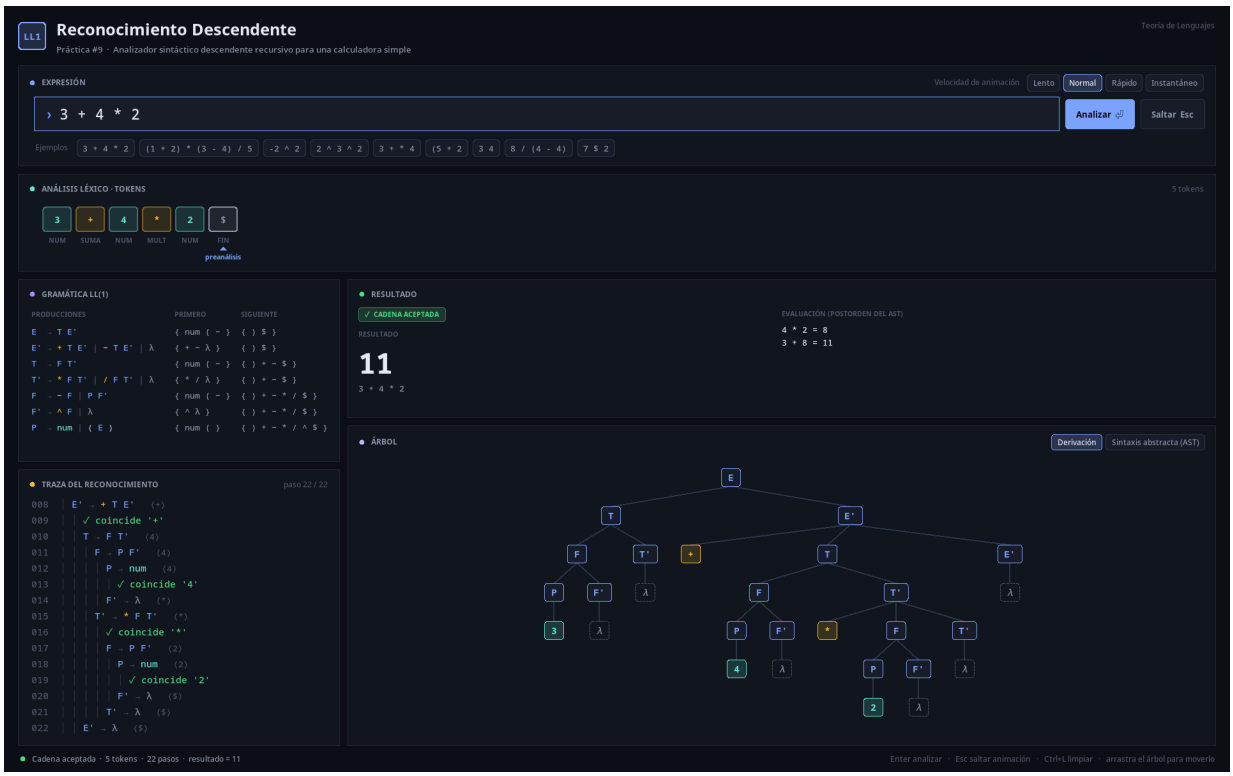
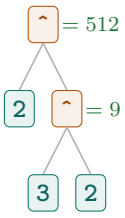
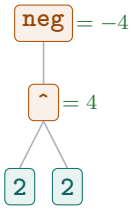


Figura 8: Resultado en la interfaz: cadena aceptada, valor 11, pasos de evaluación y árbol de derivación completo.

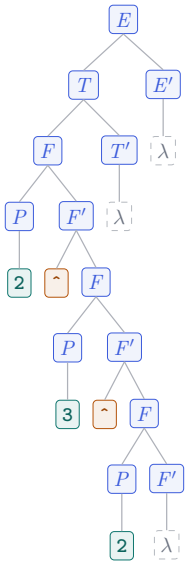
6.2 Ejemplo 2: asociatividad de la potencia y menos unario



(a) `2 ^ 3 ^ 2` $= 2^{(3^2)} = 512$



(b) `-2 ^ 2` $= -(2^2) = -4$

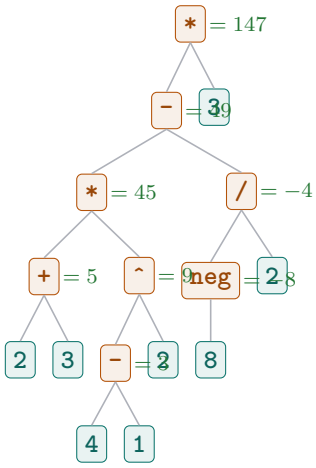


(c) Derivación de `2 ^ 3 ^ 2`

Figura 9: La potencia anida hacia la derecha porque $F' \to \texttt{\^{}}\ F$ reinicia el reconocimiento de un factor completo; el menos unario queda por encima de `^` en el AST.

6.3 Ejemplo 3: expresión compuesta

Para `((2+3)*(4-1)^2 - -8/2) * 3` el analizador realiza 84 pasos sobre 22 tokens. El AST (figura 10) muestra las ocho operaciones y sus valores intermedios.



Evaluación en postorden

```
2 + 3 = 5
4 - 1 = 3
3 ^ 2 = 9
5 * 9 = 45
-(8) = -8
-8 / 2 = -4
45 - -4 = 49
49 * 3 = 147
```

Figura 10: AST de `((2+3)*(4-1)^2 - -8/2) * 3` $= 147$.

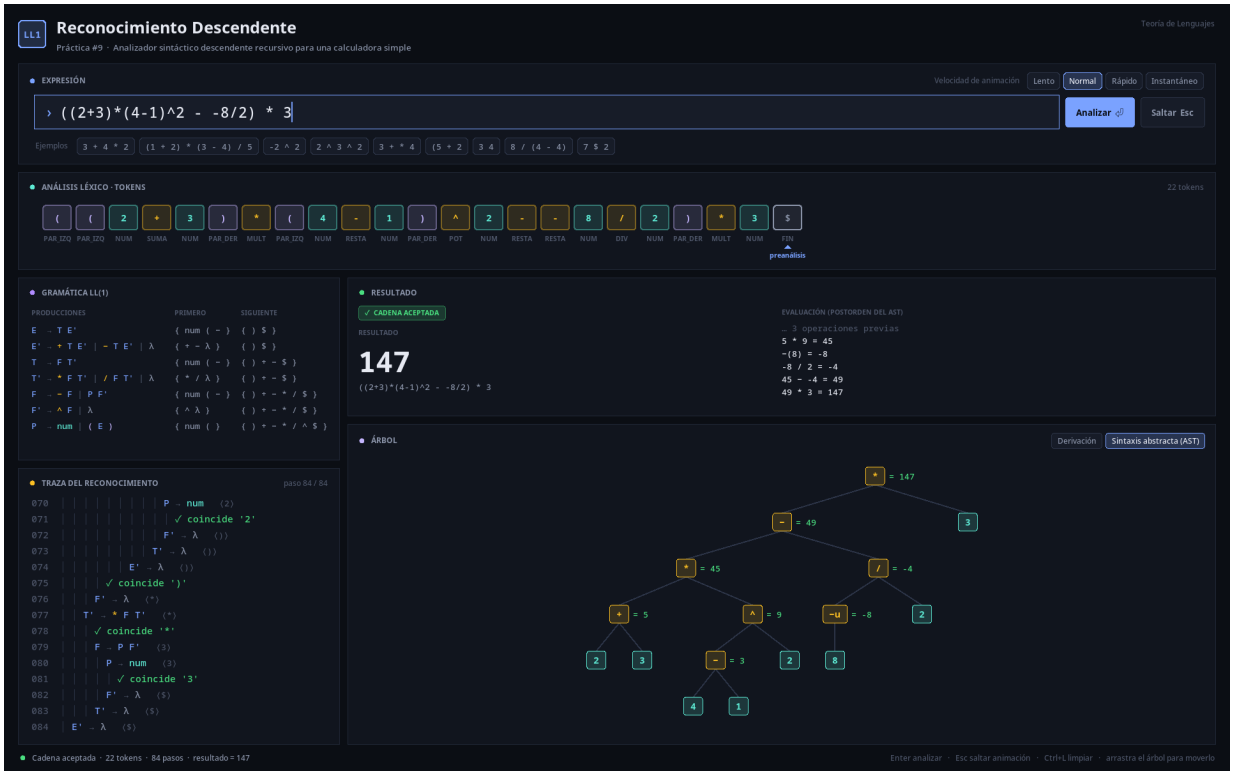


Figura 11: Vista AST en la interfaz para la misma expresión.

6.4 Ejemplo 4: error sintáctico `3 + * 4`

Tras emparejar `+`, E' llama a `T()`, éste a `F()` y éste a `P()`. El preanálisis es `*`, que no pertenece a $\textsc{Primero}(P) = \{\texttt{num}, \texttt{(}\}$: falta un operando. El error se detecta en el *primer* token que no puede extender un prefijo válido, que es la propiedad del prefijo viable de los analizadores LL(1).

Tabla 9: Traza de `3 + * 4`: el último paso es el diagnóstico.

#	Nivel	Acción	Preanálisis	Restante
1	0	$E \to T\ E'$	3	3 + * 4 $
2	1	$T \to F\ T'$	3	3 + * 4 $
3	2	$F \to P\ F'$	3	3 + * 4 $
4	3	$P \to \texttt{num}$	3	3 + * 4 $
5	4	✓ coincide 3	3	+ * 4 $
6	3	$F' \to \lambda$	+	+ * 4 $
7	2	$T' \to \lambda$	+	+ * 4 $
8	1	$E' \to +\ T\ E'$	+	+ * 4 $
9	2	✓ coincide +	+	* 4 $
10	2	$T \to F\ T'$	*	* 4 $
11	3	$F \to P\ F'$	*	* 4 $
12	4	× Operador '*' inesperado después de '+': falta un operando	*	* 4 $

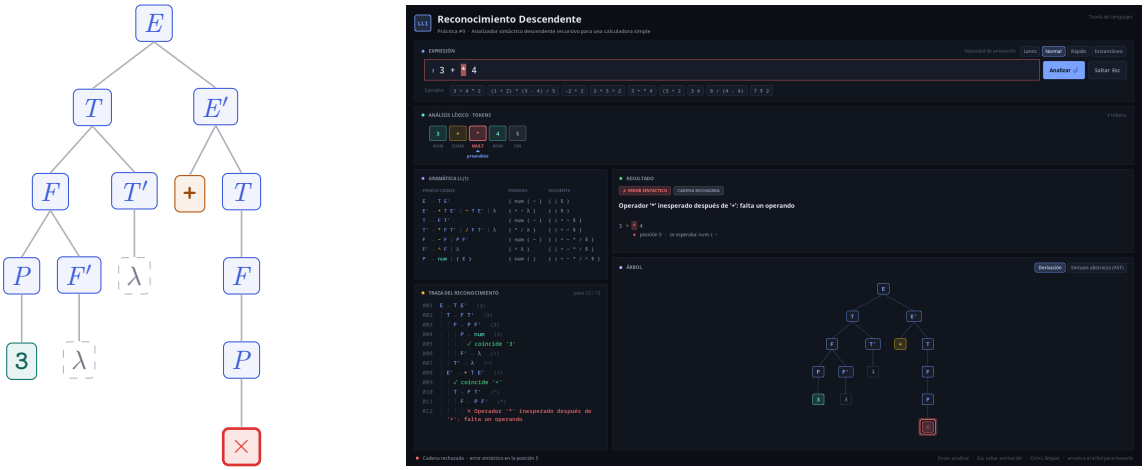
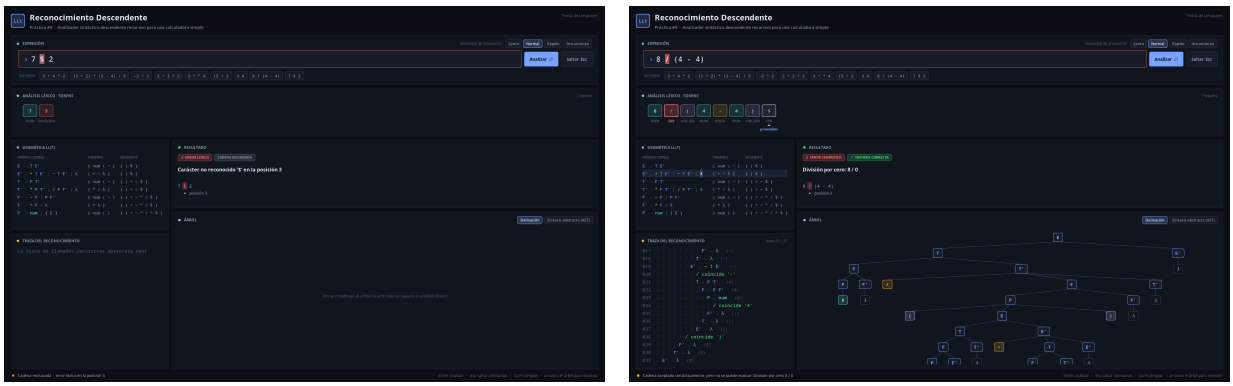


Figura 12: Árbol parcial al momento del error (nodo × bajo P) y vista de la interfaz: mensaje, posición 5, símbolos esperados (`num ( -`) y token marcado en rojo.

6.5 Ejemplo 5: errores léxico y semántico



(a) `7 $ 2`: el símbolo `$` no pertenece al alfabeto; no se inicia el análisis sintáctico.

(b) `8 / (4 - 4)`: sintaxis correcta, pero la división por cero impide evaluar.

Figura 13: Las tres clases de error se presentan con distinto distintivo; el error semántico aclara que la cadena *sí* pertenece al lenguaje.

7 Pruebas y resultados

7.1 Batería de pruebas

La tabla 10 se genera ejecutando el analizador sobre 26 entradas que cubren precedencia, asociatividad, menos unario, decimales, anidamiento, los símbolos alternativos y todos los tipos de error. En todos los casos el comportamiento obtenido coincide con el esperado.

Tabla 10: Resultados de la batería de pruebas (salida real del programa).

#	Entrada	Propósito	Veredicto	Resultado o mensaje
1	`3 + 4 * 2`	precedencia de * sobre +	aceptada	= 11
2	`(3 + 4) * 2`	paréntesis alteran la precedencia	aceptada	= 14

#	Entrada	Propósito	Veredicto	Resultado o mensaje
3	`10 - 4 - 3`	asociatividad izquierda de -	aceptada	= 3
4	`100 / 10 / 5`	asociatividad izquierda de /	aceptada	= 2
5	`2 ^ 3 ^ 2`	asociatividad derecha de ^	aceptada	= 512
6	`-2 ^ 2`	^ precede al menos unario	aceptada	= -4
7	`2 ^ -1`	exponente negativo	aceptada	= 0.5
8	`-3`	menos unario anidado	aceptada	= 3
9	`3.5 * .5 + 1`	números decimales	aceptada	= 2.75
10	`((((7))))`	paréntesis anidados	aceptada	= 7
11	`(1 + 2) * (3 - 4) / 5`	expresión compuesta	aceptada	= -0.6
12	`3 × 4 ÷ 2 - 1`	símbolos tipográficos	aceptada	= 5
13	*(vacía)*	entrada vacía	sintáctico	La expresión está vacía (pos. 1)
14	`3 + * 4`	operador sin operando	sintáctico	Operador '*' inesperado después de '+': falta un operando (pos. 5)
15	`2 +`	expresión incompleta	sintáctico	Expresión incompleta: falta un operando después de '+' (pos. 4)
16	`* 3`	inicia con operador binario	sintáctico	La expresión no puede comenzar con el operador '*' (pos. 1)
17	`3 4`	falta operador	sintáctico	Falta un operador entre '3' y '4' (pos. 3)
18	`3 (4)`	falta operador antes de (	sintáctico	Falta un operador entre '3' y '(' (pos. 3)
19	`(5 + 2`	paréntesis sin cerrar	sintáctico	Falta ')' para cerrar el '(' de la posición 1
20	`5 + 2)`	paréntesis sin abrir	sintáctico	Paréntesis ')' sin abrir en la posición 6
21	`()`	paréntesis vacíos	sintáctico	Paréntesis vacíos: se esperaba una expresión dentro de '( )' (pos. 2)
22	`7 $ 2`	carácter inválido	léxico	Carácter no reconocido '$' en la posición 3
23	`1.5.2`	número mal formado	léxico	Número mal formado: '1.5.2' (pos. 1)
24	`8 / (4 - 4)`	división por cero	semántico	División por cero: 8 / 0 (pos. 3)
25	`(-8) ^ 0.5`	potencia no real	semántico	Potencia no real: -8 ^ 0.5 (pos. 6)
26	`9 ^ 9 ^ 9`	desbordamiento	semántico	Desbordamiento: el resultado es demasiado grande (pos. 3)

7.2 Costo del reconocimiento

Cada token se empareja exactamente una vez y entre dos emparejamientos solo hay un número acotado de expansiones (a lo sumo la cadena $E \to T \to F \to P$ y los cierres λ), por lo que el número de pasos crece *linealmente* con el número de tokens: el analizador es $O(n)$ en tiempo y,

por la pila de recursión, $O(d)$ en memoria, con d la profundidad de anidamiento. La figura 14 lo confirma empíricamente sobre 140 expresiones válidas generadas al azar.

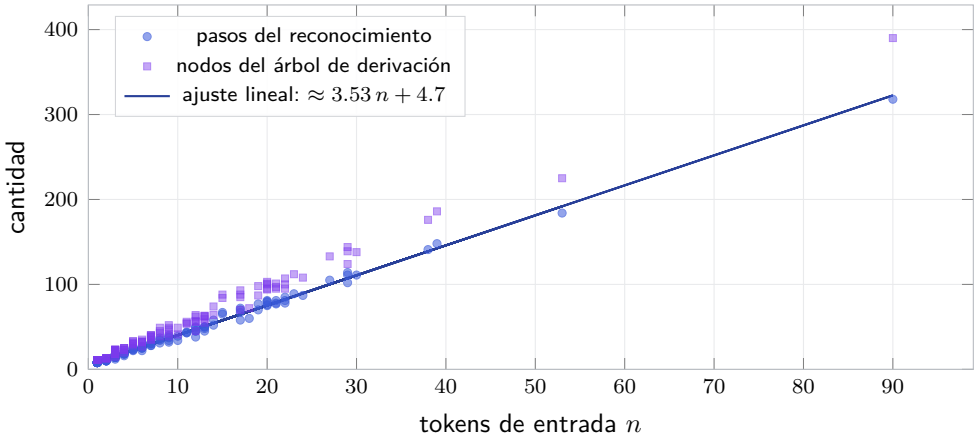


Figura 14: Pasos del analizador y tamaño del árbol de derivación frente al número de tokens: ambos siguen una recta, confirmando el costo lineal del análisis LL(1).

8 Manual de uso

Requisitos. Python 3.10 o superior con `tkinter` (incluido en las instalaciones estándar de Windows y macOS; en Linux, paquete `python3-tk` o `tk`). No hay dependencias externas.

```
# interfaz gráfica
$ python3 app.py

# modo consola: tokens, traza, evaluación y resultado
$ python3 analizador.py "3 + 4 * 2"

# regenerar tablas, trazas y árboles de este informe
$ cd informe && python3 generar.py && pdflatex informe.tex
```

Tabla 11: Controles de la interfaz gráfica.

Control	Acción
`Enter` / Analizar	Analiza la expresión escrita y anima el reconocimiento.
`Esc` / Saltar	Completa la animación en curso de inmediato.
`Ctrl+L`	Limpia la entrada y todos los paneles.
Ejemplos	Nueve expresiones de un clic, válidas e inválidas.
Velocidad	Lento, Normal, Rápido o Instantáneo (cambiable durante la animación).
Derivación / AST	Alterna entre el árbol de derivación y el árbol de sintaxis abstracta.
Arrastrar / rueda	Desplaza el árbol cuando no cabe en el panel.

9 Conclusiones

- La gramática natural de una calculadora es ambigua; estratificarla por precedencia, eliminar la recursividad por la izquierda y factorizarla produce una gramática LL(1) equivalente, cuya tabla de análisis se verificó automáticamente sin conflictos.
- El descenso recursivo traduce esa gramática a código de manera casi mecánica —un procedimiento por no terminal— y su traza coincide paso a paso con la derivación por la izquierda de la cadena.
- La recursividad por la derecha introducida por la transformación no altera la semántica: un atributo heredado permite construir un AST con la asociatividad izquierda correcta, mientras que la potencia conserva su asociatividad derecha.
- Elegir λ únicamente cuando el preanálisis está en SIGUIENTE hace que los errores se reporten en el primer token inválido, con mensajes precisos y la lista de símbolos esperados.
- Separar errores léxicos, sintácticos y semánticos aclara al usuario si su cadena pertenece o no al lenguaje, aun cuando no pueda evaluarse.
- El análisis es lineal en el tamaño de la entrada, como se comprobó experimentalmente; y la visualización animada convierte un proceso abstracto en uno observable, útil como herramienta didáctica.

Referencias

[1] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman. *Compiladores: principios, técnicas y herramientas*, 2.ª ed. Pearson, 2008. Cap. 4: Análisis sintáctico.

[2] J. E. Hopcroft, R. Motwani y J. D. Ullman. *Introducción a la teoría de autómatas, lenguajes y computación*, 3.ª ed. Pearson, 2007.

[3] K. D. Cooper y L. Torczon. *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011. Cap. 3.

[4] N. Wirth. *Compiler Construction*. Addison-Wesley, 1996. Cap. 4: Análisis descendente recursivo.

[5] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html

A Cálculo de PRIMERO y SIGUIENTE

analizador.py — calcular_primero_siguiente()

```python
def calcular_primero_siguiente(g=GRAMATICA):
    """Conjuntos PRIMERO y SIGUIENTE calculados por punto fijo."""
    primero = {a: set() for a, _ in g}
    cambio = True
    while cambio:
        cambio = False
        for a, alts in g:
            for alt in alts:
                nuevo = _primero_cadena(alt, primero)
                if not nuevo <= primero[a]:
                    primero[a] |= nuevo
```

```
                    cambio = True
    siguiente = {a: set() for a, _ in g}
    siguiente[g[0][0]].add("$")
    cambio = True
    while cambio:
        cambio = False
        for a, alts in g:
            for alt in alts:
                for i, b in enumerate(alt):
                    if b not in siguiente:
                        continue
                    resto = _primero_cadena(alt[i + 1:], primero)
                    nuevo = (resto - {"λ"}) | (siguiente[a] if "λ" in resto else set())
                    if not nuevo <= siguiente[b]:
                        siguiente[b] |= nuevo
                        cambio = True
    ordenar = lambda s: sorted(s, key=_ORDEN.index)
    return ({a: ordenar(v) for a, v in primero.items()},
            {a: ordenar(v) for a, v in siguiente.items()})
```